

Push-Benachrichtigungen

Was steht, was fehlt und wie du es besorgst · Stand 25. September 2026

So liest du das: **Rot** musst du selbst machen — es hängt an deinen Konten. **Grün** ist erledigt. **Gelb** heißt: angefangen oder von hier aus nicht nachprüfbar. **Der Server-Teil ist seit dem 24.09.2026 fertig und läuft** — offen ist nur noch, was Apple-Konten und Stores betrifft.

Die Übersicht

#	TEIL	STATUS	WER
1	Bezahlte Apple-Mitgliedschaft im Team <code>HACJC6623Z</code> auf diesem Mac	fehlt	du
2	Push-Berechtigung für die App-ID <code>app.matchup.mobile</code>	ungeprüft	du
3	Key ID des APNs-Schlüssels — <code>9JF35BFS9Q</code>	da	—
4	Schlüsseldatei <code>AuthKey_9JF35BFS9Q.p8</code> in <code>~/keys/</code>	da	—
5	Team ID — <code>HACJC6623Z</code>	da	—
6	Bundle ID — <code>app.matchup.mobile</code>	da	—
7	Firebase-Projekt <code>matchup-f9e83</code> , Analytics aus	da	—
8	<code>GoogleService-Info.plist</code> + <code>google-services.json</code>	im Projekt	—
9	Dienstkonto-Schlüssel als Secret <code>FIREBASE_DIENSTKONT0</code>	gesetzt	—
10	APNs-Schlüssel bei Firebase hochgeladen	sagst du mir	du
11	Datenschutzerklärung veröffentlicht · Store-Angaben	Seite live, Stores offen	du
12	<code>aps-environment</code> , Berechtigungsabfrage, Geräte-Tokens	fertig	
13	Tabellen, Auslöser und Einstellungen je Nutzer	Migration 0129	
14	Edge Function <code>push</code> — ausgespielt, drei Cron-Jobs aktiv	läuft	

Am 24.09.2026 nachgemessen, nicht behauptet: Die Tabellen `push_geraete`, `push_einstellungen` und `push_auftraege` stehen, die Function antwortet `{"auftraege":0,"gesendet":0}`, und ihr Minutenlauf kommt mit `200` am Gateway vorbei (kein 401). Unverändert offen: `security find-identity` kennt weiterhin nur ein *Apple Development*-Zertifikat, kein *Distribution*-Zertifikat.

Vorher lesen — drei Regeln

- Die **Key ID** (`9JF35BFS9Q`) ist keine Geheimnummer, die darfst du weitergeben.
- Die **Datei** `.p8` **ist das Geheimnis**. Ihren Inhalt nirgends hineinkopieren — auch nicht in einen Chat. Wenn ich sie brauche, sag mir den **Dateipfad**.
- Android geht ohne Apple**. Zieht sich Schritt 1, machst du nur Schritt 4, 5 und 7 — dann baue ich Push für Android fertig, iOS kommt später dazu. Der Server-Teil ist derselbe.

1 Apple-Mitgliedschaft auf diesem Mac einrichten

DU · 10 MIN

Warum: Ohne bezahltes Team darf die App keine Push-Benachrichtigungen verwenden. Ein persönliches Team reicht nicht — derselbe Grund, aus dem `flutter build ipa` scheitert.

1. Xcode öffnen.
2. Menü **Xcode** → **Settings...** (oder  + ).
3. Reiter **Accounts**.
4. Unten links auf + → **Apple ID** → **Continue**.
5. Mit der Apple ID anmelden, die zum bezahlten Team **HACJC6623Z** gehört.
6. Links das Konto anklicken. Rechts muss **MatchUp** (**HACJC6623Z**) stehen — Rolle *Account Holder*, *Admin* oder *App Manager*.
7. Rechts unten **Download Manual Profiles** klicken.

```
security find-identity -v -p codesigning
```

Fertig, wenn dort eine Zeile mit *Apple Distribution* steht. Steht weiterhin nur *Apple Development*, bist du im falschen Team angemeldet oder die Mitgliedschaft ist abgelaufen.

2 Push für die App-ID einschalten

DU · 3 MIN

Warum: Apple muss wissen, dass genau diese App Push benutzen darf.

1. `developer.apple.com/account` öffnen, anmelden.
2. Oben rechts prüfen: Steht dort **MatchUp / HACJC6623Z**? Sonst umschalten.
3. Links **Certificates, Identifiers & Profiles** → **Identifiers**.
4. **app.matchup.mobile** anklicken.
5. Häkchen bei **Push Notifications** setzen.
6. Oben rechts **Save**, Rückfrage mit **Confirm** bestätigen.

Fertig, wenn neben *Push Notifications* ein Häkchen steht und die Seite *Enabled* anzeigt.

3 Die Schlüsseldatei .p8 besorgen

ERLEDIGT

Steht seit dem 24.09.2026: ~/keys/AuthKey_9JF35BFS9Q.p8 . Die Anleitung bleibt stehen, falls der Schlüssel je ersetzt werden muss.

Warum: Sie ist der eigentliche Schlüssel. Key ID und Team ID sind nur die Kennnummern dazu.

Erst suchen. Die Datei heißt AuthKey_9JF35BFS9Q.p8 :

```
find ~ -name "AuthKey_*.p8" 2>/dev/null
```

Taucht sie auf: nach ~/keys/ legen (nicht ins Projekt) — fertig, weiter bei Schritt 4.

Wenn sie weg ist: Apple gibt eine .p8 nur **einmal** heraus. Dann legst du einen neuen Schlüssel an — unkritisch, den alten benutzt noch nichts.

1. `developer.apple.com/account` → **Certificates, Identifiers & Profiles** → links **Keys**.
2. Den alten Schlüssel 9JF35BFS9Q anklicken → **Revoke** → bestätigen.
3. Oben auf + (Create a key).
4. **Key Name:** MatchUp Push .
5. Häkchen bei **Apple Push Notifications service (APNs)**.
6. **Continue** → **Register**.
7. **Download** klicken — landet in ~/Downloads .
8. **Neue Key ID notieren** (steht auf der Seite und im Dateinamen). Sag sie mir.

```
mkdir -p ~/keys && mv ~/Downloads/AuthKey_*.p8 ~/keys/
```

Fertig, wenn `ls ~/keys/AuthKey_*.p8` die Datei anzeigt.

4 Firebase-Projekt anlegen

ERLEDIGT

Steht seit dem 21.09.2026: Projekt matchup-f9e83 , Spark-Tarif, Analytics aus.

Warum: Firebase Cloud Messaging verschickt an iOS **und** Android über einen Weg. Ohne Firebase bräuchte es zwei Versandwege. Kostet nichts.

1. `console.firebase.google.com` öffnen, mit Google-Konto anmelden.
2. **Projekt hinzufügen**.
3. Name: MatchUp → **Weiter**.
4. Google Analytics **ausschalten** (brauchen wir nicht, spart eine Datenschutz-Baustelle) → **Projekt erstellen**.
5. Warten, dann **Weiter**.

Fertig, wenn du die Projektübersicht siehst.

5 Die beiden Apps in Firebase eintragen

ERLEDIGT

Beide Konfigdateien liegen im Projekt: `android/app/google-services.json` und `ios/Runner/GoogleService-Info.plist`.

Android

1. In der Projektübersicht auf das **Android-Symbol**.
2. **Paketname:** `app.matchup.mobile` — genau so, ohne Leerzeichen.
3. Spitzname `MatchUp Android`, SHA-1 leer lassen → **App registrieren**.
4. **google-services.json herunterladen**.
5. Die Schritte „Firebase SDK hinzufügen“ **überspringen** — das mache ich. **Weiter** → **Weiter zur Konsole**.

```
mv ~/Downloads/google-services.json ~/Projekte/MatchUp/android/app/
```

iOS

1. **App hinzufügen** → **iOS**.
2. **Bundle-ID:** `app.matchup.mobile`.
3. Spitzname `MatchUp iOS`, App Store ID leer → **App registrieren**.
4. **GoogleService-Info.plist herunterladen**, Rest überspringen.

```
mv ~/Downloads/GoogleService-Info.plist ~/Projekte/MatchUp/ios/Runner/
```

Fertig, wenn beide Dateien an ihrem Platz liegen: `ls ~/Projekte/MatchUp/android/app/google-services.json` und `ls ~/Projekte/MatchUp/ios/Runner/GoogleService-Info.plist`

6 Den APNs-Schlüssel bei Firebase hinterlegen

OFFEN · DU

Das ist der eine Schritt, den ich nicht nachsehen kann. Die Firebase-Konsole verrät mir nicht, ob der Schlüssel dort liegt — sag mir, ob er steht. Ohne ihn nimmt Apple die Nachrichten nicht an, auch wenn alles andere läuft.

Warum: Firebase spricht in deinem Namen mit Apple. Dafür braucht es die `.p8` samt Kennnummern.

1. Firebase-Konsole → Zahnrad oben links → **Projekteinstellungen**.
2. Reiter **Cloud Messaging**.
3. Bei `app.matchup.mobile`: **APNs-Authentifizierungsschlüssel** → **Hochladen**.
4. **Datei:** die `.p8` aus `~/keys/`.
5. **Key ID:** `9JF35BFS9Q`.
6. **Team ID:** `HACJC6623Z`.
7. **Hochladen**.

Fertig, wenn der Schlüssel mit seiner Key ID in der Liste steht.

7 Dienstkonto-Schlüssel für den Versand

ERLEDIGT

Steht. Die Datei liegt als `~/keys/firebase-dienstkonto.json` und seit dem 24.09.2026 als Supabase-Secret `FIREBASE_DIENSTKONTO` — im Klartext ist sie nirgends aufgetaucht.

Warum: Unser Server (die Edge Function) muss sich bei Firebase ausweisen, um Nachrichten zu verschicken.

1. Firebase-Konsole → **Projekteinstellungen** → Reiter **Dienstkonten**.
2. **Neuen privaten Schlüssel generieren** → **Schlüssel generieren**.
3. Die JSON-Datei landet in `~/Downloads`.

```
mv ~/Downloads/matchup*-firebase-adminsdk-*.json ~/keys/firebase-dienstkonto.json
```

Vollzugriff-Schlüssel. Nicht weitergeben, nicht ins Repo. Sag mir nur den Pfad — ich lege ihn als Supabase-Secret ab, ohne ihn anzuzeigen.

Fertig, wenn `ls ~/keys/firebase-dienstkonto.json` die Datei zeigt.

8 Datenschutz

SEITE LIVE · STORES OFFEN

Am 25.09.2026 veröffentlicht: sleeperdach.github.io/datenschutz.html zeigt Abschnitt 6 („Push-Benachrichtigungen“), die Quelle liegt als `docs/datenschutz.html` im Projekt. Beim Übernehmen wurde der `<!DOCTYPE html>` ergänzt — die Fassung begann direkt mit `<html>`, was Browser in den Quirks-Modus schickt; am Wortlaut wurde nichts geändert. **Offen bleiben nur die beiden Store-Angaben.**

Warum: Firebase ist ein Google-Dienst und verarbeitet eine Geräte-Kennung. Das muss in der App und in beiden Stores stehen.

- **Datenschutzerklärung** im Impressum ergänzen: Firebase Cloud Messaging (Google Ireland Ltd.), Zweck Push-Benachrichtigungen, Daten: Geräte-Token.
- **App Store Connect** → App-Datenschutz: „Kennungen → Geräte-ID“ angeben.
- **Google Play Console** → Datensicherheit: dasselbe.

Sag Bescheid, wenn du dabei Formulierungen brauchst — die schreibe ich dir.

Was ich gemacht habe

Alles davon steht und ist nachgemessen (24.09.2026):

- **App:** `firebase_messaging` einbinden, Berechtigung abfragen (iOS beim ersten Start, Android ab Version 13), `aps-environment` und Hintergrundmodus in die iOS-Einstellungen, Antippen öffnet den passenden Schirm.
- **Datenbank:** Tabelle für Geräte-Tokens je Nutzer mit RLS, Einstellungen je Kategorie; ungültige Tokens räumt der Versand selbst weg.
- **Edge Function** `push` : verschickt über Firebase, ausgelöst von DB-Triggern bzw. den bestehenden Crons.
- **Einstellungsschirm** mit einem Schalter je Kategorie, dazu Tests.

Zum Ausprobieren brauchst du ein echtes iPhone. Im Simulator ist Push mit Firebase unzuverlässig. Android geht auch im Emulator.

Der Server läuft – nachgemessen

WAS	NACHGEMESSEN AM 24.09.2026
Migration 0129	<code>push_geraete</code> , <code>push_einstellungen</code> , <code>push_auftraege</code> stehen
Secret	<code>FIREBASE_DIENSTKONTO</code> wird von <code>supabase secrets list</code> geführt
Function	ausgespielt mit <code>--no-verify-jwt</code> , antwortet <code>{"auftraege":0,"gesendet":0}</code>
Cron	<code>push-versand</code> (jede Minute), <code>push-tipp-erinnerungen</code> (alle 15 Min), <code>push-korb-aufräumen</code> (04:17) — alle drei aktiv
Auslöser	10 Trigger an Chats, Trades, Waivern, Ausfällen und Anfragen
Gateway	der Minutenlauf steht mit <code>200</code> in <code>net._http_response</code> — kein 401

Dass `gesendet` auf 0 steht, ist kein Fehler: Solange kein Gerät einen Token gemeldet hat, bleibt der Korb leer. Dafür braucht es Schritt 1, 2 und 6 — und ein echtes iPhone.

Was Push auslösen soll – Vorschlag

EREIGNIS	QUELLE
Du bist im Draft am Zug	<code>draft_picks</code>
Trade-Angebot: neu, angenommen, abgelehnt	<code>fantasy_trades</code>
Waiver durchgegangen oder nicht	<code>fantasy_waiver_claims</code>
Direktnachricht, Liga- oder Tipprunden-Chat	<code>direct_messages</code> , <code>*_messages</code>
Aufgestellter Spieler fällt aus	<code>player_absences</code>
Tipps offen, kurz vor Deadline	<code>tip_rounds</code>

EREIGNIS**QUELLE**

Beitritts- oder Freundschaftsanfrage

`*_join_requests` , `friendships`

Alle diese Ereignisse gibt es bereits als Realtime-Tabellen. Sag mir, womit die erste Fassung anfangen soll.

MatchUp · Push-Benachrichtigungen · Stand 25.09.2026 · Dieselben Inhalte stehen als `PUSH.md` im Projektverzeichnis.